replace it with a managed service offered by a cloud platform. Then we'll add a data persistence layer to Catalog Service using Spring Data JDBC. Finally, I'll cover some common concerns about managing and evolving databases in production with Flyway.

NOTE The source code for the examples in this chapter is available in the Chapter05/05-begin, Chapter05/05-intermediate, and Chapter05/05-end folders on GitHub, containing the initial, intermediate, and final states of the project (https://github.com/ThomasVitale/cloud-native-spring-in-action).

5.1 Databases for cloud native systems

Data can be stored in many ways. Traditionally you would probably use a single massive database server to save as much as possible, since getting a new one would be expensive and time-consuming. Depending on your organization's processes, a task like that could take from a few days to several months. Not in the cloud.

The cloud offers elastic, self-service, and on-demand provisioning features —strong motivators for migrating your data services there. For each cloud native application you design, you should consider the most suitable storage type for the data it will produce. Then your cloud platform should let you provision it through an API or a graphical user interface. A task that used to be very time-consuming now requires only a few minutes. For example, deploying an instance of a PostgreSQL database server on Azure is as simple as running the `az postgres server create` command.

Cloud native applications are designed to be stateless because of the nature of the cloud itself. It's a dynamic infrastructure where the computing nodes can spread across different clusters, geographical regions, and clouds. The problem of applications storing state is evident. How could the state survive